Lacy Hamilton 3/31/2026

Halma – Part 0

Overview

The architecture I chose to implement for Halma part zero was to have two classes, *Logic* and *Halma*.

The *Logic* class, as it is named, controls the actual game logic which is based on a 2D array of either 0 for empty, and 1 for the pieces. It initializes the array and sets up the one players position at the top right corner in a triangle position. Some of the key methods are:

- *get_adjacent_moves()* which checks all of the 8 directions that a piece can move to see if there is a 0 or an open position, then it adds this to a list of moves that can be made.
- *get_all_jump_moves()* is a recursive method which checks the directions for a piece to jump over. To do this it basically sees if the 'mid' position has a piece, and if the position after or the position to jump to is open. This would allow a valid jump. It recursively does this to see if there are chained jumps available. Also added a check to make sure we stay inside the bounds of the board. Returns the list of the positions we could jump to.
- *move_piece()* moves the piece by changing the position we are moving 'from' to zero and the new position we move 'to' to a one. Zero represents an open position and the one shows a piece is there. This allows us to use the GUI methods to redraw the board with the new positions.

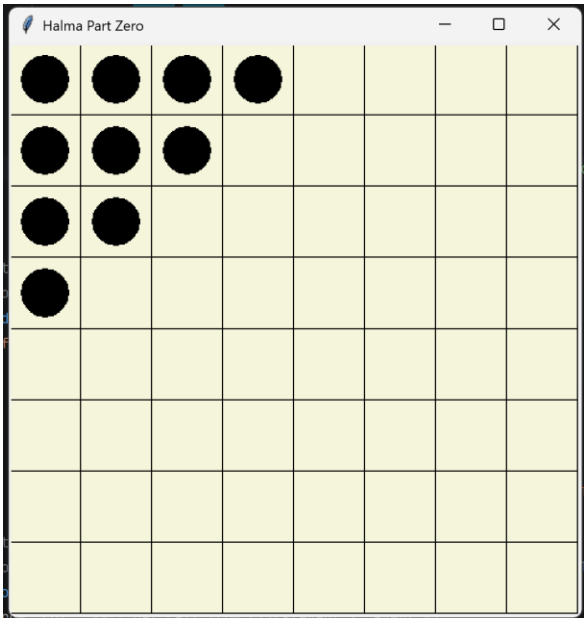
The *Halma* class, controls the actual GUI and clicking aspects of the game. It initializes the window and the canvas root for the *tkinter* library and sets up the settings like the width, and height based on what I wanted the cell size to be, attaches to the Logic class to implement the array in a graphical manner, and also binds the mouse click so we can track it. The class has a *mainloop()* and it has these key methods:

- *draw_grid()* creates the grid for the window using create_rectangle based on the cell size.
- *draw_pieces()* loops through the board and adds a black piece where the position is one, also will highlight the piece in pink and red if it is the selected position.
- *draw_highlights()* highlights the valid moves in green using *create_rectangle*
- *on_click()* is the driver for the clicking, it calls *get_cell_from_click* to get the actual position (row, col), moves the piece if its open, or highlights the selected piece.
- *refresh_window()* is a function which refreshed the full board for each change. It deletes the positions using *canvas.delete* then redraws the grid, draws the valid moves highlights, and then adds the pieces again.

Lacy Hamilton 3/31/2026

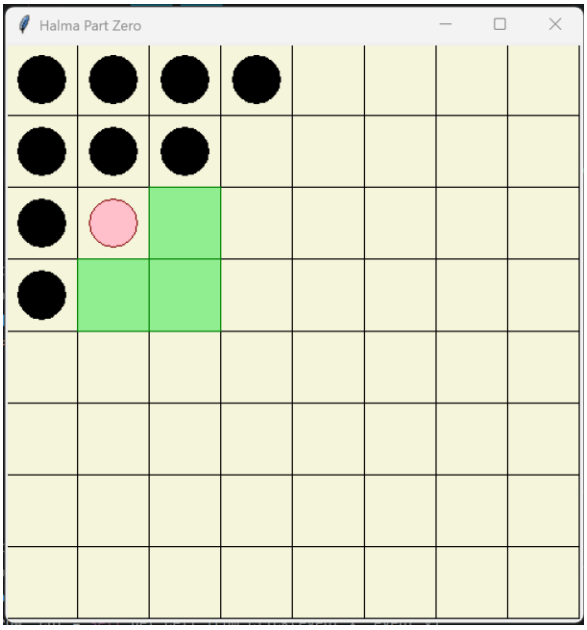
Display 8x8 Initial Grid

Initial grid set to an 8 by 8, with one player's pieces in black on the top-right corner.



Possible Moves

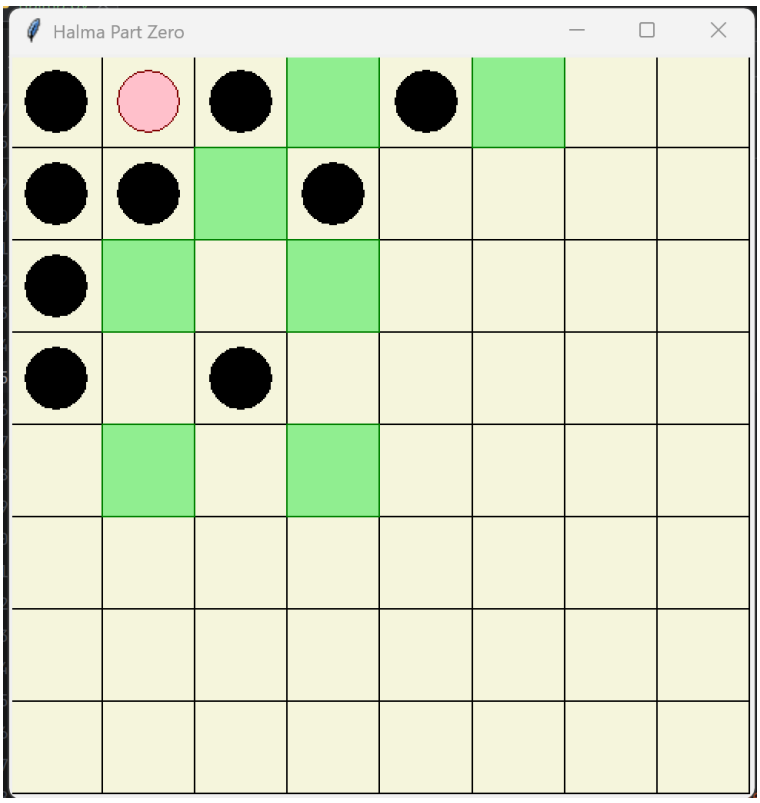
This screenshot shows me clicking on piece which highlights pink with red outline. It then shows the possible moves by highlighting the square in green and light green fill using *create_rectangle*.



Lacy Hamilton 3/31/2026

Moved Pieces

This screenshot is after moving some pieces. You can see that the selected piece is higher up and due to some of the pieces being moved it shows the options for chained jumps. When the pieces are moved, it deletes where it was using by changing the board logic to zero erasing everything, then adding pieces and the highlights accordingly to the provided lists (*valid_moves* and the positions for pieces).

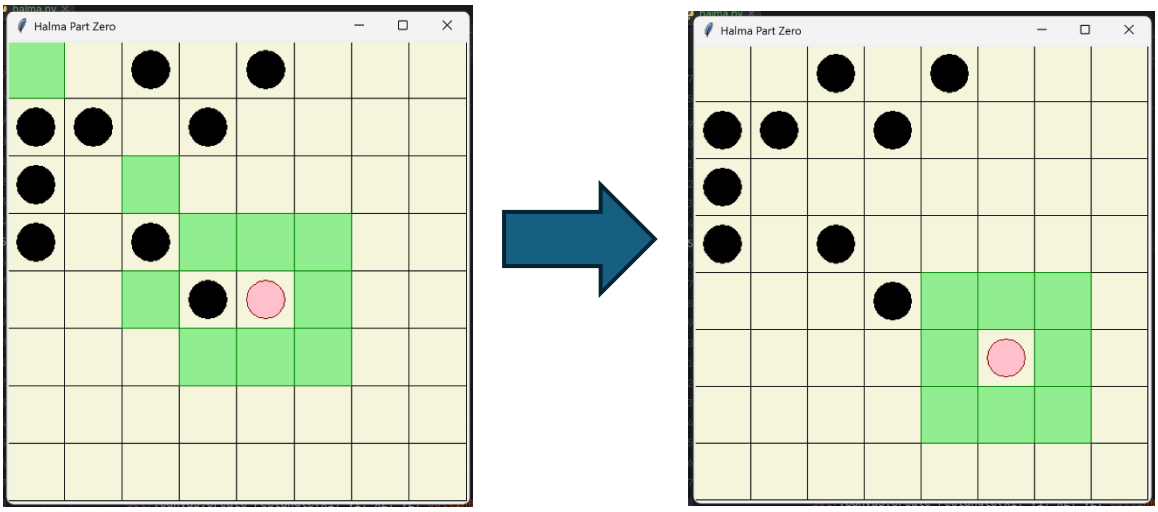


Lacy Hamilton 3/31/2026

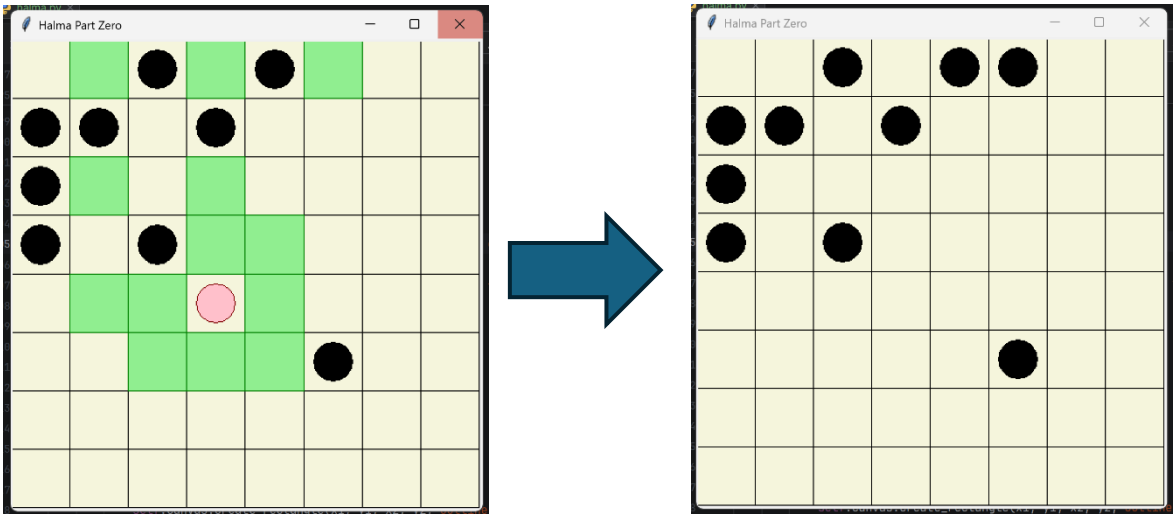
Moves Available after Several Consecutive Jumps

Implementing moves to adjacent squares shows the available jumping and chained jumping. I provided two examples where I move one away from the ability to chain. With the second example I show a piece actually chaining a jump down to a much further position.

Example 1:



Example 2:



Lacy Hamilton 3/31/2026

Code

```
# halma part 0
# process: halma is a two-player, turn-based board game played on a square grid

# header files
import tkinter as tk

# constants
CELL_SIZE = 60

# ========================================================================== #
#                                GAME LOGIC                                  #
# ========================================================================== #

# class: Logic
# process: logic for the game process
class Logic:

    # method: init
    # process: creates 2D array, places the pieces in the corners
    def __init__(self, size):
        self.size = size

        # create the 2D array, initialize all spaces to zero
        self.board = [[0 for x in range(size)] for y in range(size)]

        # halves the board size for the pieces initialization
        half = size // 2

        # creates the top triangle
        for row in range(half):
            for col in range(half - row):
                self.board[row][col] = 1

        # # creates the bottom triangle (not for part 0 add probs in part 1)
        # for row in range(half, self.size):
        #     start_column = self.size - 1 - (row - half)
        #     for col in range(start_column, self.size):
        #         self.board[row][col] = 1


    # method: move_piece
    # process: update the board array
    def move_piece(self, from_position, to_position):

        # set variables for the coords from and to
        from_row, from_col = from_position
        to_row, to_col = to_position

        # set the old value to zero, new to one
        self.board[from_row][from_col] = 0
        self.board[to_row][to_col] = 1

    # method: get_adjacent_moves
    # process: check all 8 directions, return empty places
    def get_adjacent_moves(self, row, col):

        # initialize variables
        moves = []
        directions = [(-1,-1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1,
1)]
```

Lacy Hamilton 3/31/2026

```
        # traverse the directions
        for x, y in directions:
            # assign the variables with the next row and col
            next_row, next_col = row + x, col + y

            # append any positions that are empty to list of moves
            if 0 <= next_row < self.size and 0 <= next_col < self.size:
                if self.board[next_row][next_col] == 0:
                    moves.append((next_row, next_col))

        # return list of moves
        return moves

    # method: get_all_jump_moves
    # process: recursive function, made to capture potential chained jumps, and simple
jumps
    def get_all_jump_moves(self, row, col, visited=None):

        # base case for the recursion
        if visited is None:
            visited = set()

        # initialize variables
        directions = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0),
(1, 1)]
        moves = set()

        # traverse all the directions
        for x, y in directions:
            # grab the mid positional value
            mid_row, mid_col = row + x, col + y

            # grab the position we would be jumping to
            jump_row, jump_col = row + 2 * x, col + 2 * y

            # check bounds of the board
            if not (0 <= jump_row < self.size and 0 <= jump_col < self.size):
                continue

            # check that it would be a valid jump (mid needs a piece jump needs to be
open)
            if self.board[mid_row][mid_col] == 1 and self.board[jump_row][jump_col] ==
0:

               # if the jump has not been visited
                if (jump_row, jump_col) not in visited:

                    # add it to the potential moves
                    moves.add((jump_row, jump_col))

                    # mark this visit before recursion
                    new_visited = visited.copy()
                    new_visited.add((jump_row, jump_col))

                    # now call the function to recurse
                    chained = self.get_all_jump_moves(jump_row, jump_col, new_visited)

                    # update moves based on chained
                    moves.update(chained)

        # return the valid chained moves
        return moves
```

Lacy Hamilton 3/31/2026

```
    # method: get_valid_moves
    # process: call functions to get all valid moves
    def get_valid_moves(self, row, col):

        # makes the adjacent moves list
        adjacent_moves = self.get_adjacent_moves(row, col)

        # gets all the available jumps in a list (comes in as a set)
        jump_moves = list(self.get_all_jump_moves(row, col))

        # return both lists
        return adjacent_moves + jump_moves

    # temporary testing for the board logic
    def print_board(self):
        for row in self.board:
            print(row)
        print()

# ========================================================================== #
#                                GUI                                         #
# ========================================================================== #

# class: Halma
# process: class for the gui and tkinter
class Halma:

    # method: init
    # process: initializes tkinter and creates the canvas and board object
    def __init__(self, size=8):
        self.size = size

        # creates the window, adds a title
        self.window = tk.Tk()
        self.window.title("Halma Part Zero")

        # creates a canvas with a width of board * cell size
        self.canvas = tk.Canvas(
            self.window,
            width = self.size * CELL_SIZE,
            height = self.size * CELL_SIZE, )
        self.canvas.pack()

        # attach the board logic
        self.board = Logic(size)

        # selection variables
        self.selected_piece = None
        self.valid_moves = []

        # binds the mouse click
        self.canvas.bind("<Button-1>", self.on_click)

        # initial draw
        self.refresh_window()

        # main loop
        self.window.mainloop()

    # method: draw_grid
    # process: draw NxN squares using create_rectangle
    def draw_grid(self):
        # traverse the NxN grid
```

Lacy Hamilton 3/31/2026

```
        for row in range(self.size):
            for col in range(self.size):

                # get the rectangle positions based on position and cell size
                x1 = col * CELL_SIZE
                y1 = row * CELL_SIZE
                x2 = x1 + CELL_SIZE
                y2 = y1 + CELL_SIZE

                # create the square for the gameboard
                self.canvas.create_rectangle(x1, y1, x2, y2, outline = "black", fill =
"beige")

    # method: draw_pieces
    # process: loop through board, if array 1 and a game piece using create_oval
    def draw_pieces(self):
        # traverse the NxN grid
        for row in range(self.size):
            for col in range(self.size):
                # if the board is supposed to have a piece (per the array)
                if self.board.board[row][col] == 1:

                    # set the values for the oval (add padding to center this)
                    x1 = col * CELL_SIZE + 10
                    y1 = row * CELL_SIZE + 10
                    x2 = x1 + CELL_SIZE - 20
                    y2 = y1 + CELL_SIZE - 20

                    # if the piece is the selected piece
                    if self.selected_piece == (row, col):
                        # highlight the piece pink
                        self.canvas.create_oval(x1, y1, x2, y2, outline = "maroon",
fill="pink")

                    # else then make the piece a black piece
                    else:
                        self.canvas.create_oval(x1, y1, x2, y2, fill="black")

    # method: draw_highlights
    # process: draws the highlights using create_rectangle
    def draw_highlights(self):
        # for each position in the valid moves
        for (row, col) in self.valid_moves:
            # grab coords for the rectangle creation based on cell size
            x1 = col * CELL_SIZE
            y1 = row * CELL_SIZE
            x2 = x1 + CELL_SIZE
            y2 = y1 + CELL_SIZE

            # create a green rectangle using create_rectangle
            self.canvas.create_rectangle(x1, y1, x2, y2, outline = "green", fill =
"lightgreen")

    # method: on_click
    # process: coverts click to board pos, if clicked piece select piece, if clicked
square move piece
    def on_click(self, event):
        # get the click event positions
        row, col = self.get_cell_from_click(event.x, event.y)

        # if the position is a valid move square
        if (row, col) in self.valid_moves:
            # move the piece in the logic
```

Lacy Hamilton 3/31/2026

```
            self.board.move_piece(self.selected_piece, (row, col))
            # unselect the piece
            self.selected_piece = None
            # reset the moves
            self.valid_moves = []
            # redraw the board
            self.refresh_window()
            # exit function
            return

        # if the position is a piece on the board
        if self.board.board[row][col] == 1:
            # make this the selected piece
            self.selected_piece = (row, col)
            # get the valid moves for the piece
            self.valid_moves = self.board.get_valid_moves(row, col)
            # redraw the board with the positions
            self.refresh_window()

    # method: get_cell_from_click
    # process: converts a click to the board position
    def get_cell_from_click(self, x, y):

        # position is the input divided by the cell size
        col = x // CELL_SIZE
        row = y // CELL_SIZE

        # return position
        return row, col

    # method: refresh_window
    # process: this is called as it clears the canvas, calls draw_grid, draw_pieces,
draw_highlights
    def refresh_window(self):
        self.canvas.delete("all")   # clears the canvas
        self.draw_grid()            # draw the grid
        self.draw_highlights()      # draw the highlights around selected piece
        self.draw_pieces()          # draw the pieces (canvas.create_oval)

# ========================================================================== #
#                                MAIN                                        #
# ========================================================================== #
if __name__ == "__main__":
    Halma(8)
```